

```

21. #include <stdio.h>
#include <string.h>

void pad(char *input, char *output, int blockSize)
{
    int len = strlen(input);
    int padding = blockSize - (len % blockSize);
    int i;

    strcpy(output, input);

    for(i = 0; i < padding - 1; i++)
        output[len + i] = '0';

    output[len + padding - 1] = '1';
    output[len + padding] = '\0';
}

int main()
{
    char plaintext[100];
    char padded[100];
    int blockSize;

    printf("Enter plaintext: ");
    scanf("%s", plaintext);

    printf("Enter block size: ");
    scanf("%d", &blockSize);

    pad(plaintext, padded, blockSize);

    printf("Original : %s\n", plaintext);
    printf("Padded   : %s\n", padded);

    printf("\nPadding is added even when the message is");
    printf(" already a complete block to distinguish");
    printf(" padding from actual plaintext during decryption.\n");

    return 0;
}

```

```

Enter plaintext: tanuja tarun
Enter block size: Original : tanuja
Padded   : tanuja000001

Padding is added even when the message is already a complete block to distinguish padding from actual plaintext during decryption.

Process returned 0 (0x0)   execution time : 4.878 s
Press any key to continue.

```

```

22. #include <stdio.h>
#include <string.h>

void xorBits(char *a, char *b, char *result, int n)
{
    int i;

    for(i = 0; i < n; i++)
        result[i] = (a[i] == b[i]) ? '0' : '1';

    result[n] = '\0';
}

void sdesEncrypt(char *input, char *key, char *output)
{
    int i;

    for(i = 0; i < 8; i++)
        output[i] = input[i];

    output[8] = '\0';

    for(i = 0; i < 8; i++)
        output[i] =
            ((output[i] - '0') ^
             (key[i % 10] - '0')) + '0';
}

void sdesDecrypt(char *input, char *key, char *output)
{
    int i;

    for(i = 0; i < 8; i++)
        output[i] = input[i];

    output[8] = '\0';

    for(i = 0; i < 8; i++)
        output[i] =
            ((output[i] - '0') ^
             (key[i % 10] - '0')) + '0';
}

int main()
{
    char plaintext[17];

```

```

char ciphertext[17];
char decrypted[17];

char block1[9], block2[9];
char cipher1[9], cipher2[9];
char temp[9];

char iv[] = "10101010";
char key[] = "0111111101";

printf("Enter 16-bit plaintext: ");
scanf("%16s", plaintext);

strncpy(block1, plaintext, 8);
block1[8] = '\0';

strcpy(block2, plaintext + 8);

xorBits(block1, iv, temp, 8);
sdesEncrypt(temp, key, cipher1);

xorBits(block2, cipher1, temp, 8);
sdesEncrypt(temp, key, cipher2);

strcpy(ciphertext, cipher1);
strcat(ciphertext, cipher2);

printf("Ciphertext : %s\n", ciphertext);

sdesDecrypt(cipher1, key, temp);
xorBits(temp, iv, block1, 8);

sdesDecrypt(cipher2, key, temp);
xorBits(temp, cipher1, block2, 8);

strcpy(decrypted, block1);
strcat(decrypted, block2);

printf("Decrypted : %s\n", decrypted);

return 0;
}

```

```
Enter 16-bit plaintext: efejfoefrekdjddwdj
Ciphertext : 10000000010000000
Decrypted  : 0101010101111111

Process returned 0 (0x0)   execution time : 4.050 s
Press any key to continue.
|
```

```
23. #include <stdio.h>
```

```
#include <string.h>
```

```
void sdesEncrypt(char *input, char *key, char *output)
```

```
{
```

```
    int i;
```

```
    for(i = 0; i < 8; i++)
```

```
        output[i] =
```

```
            ((input[i] - '0') ^
```

```
            (key[i % 10] - '0')) + '0';
```

```
    output[8] = '\0';
```

```
}
```

```
void xorBits(char *a, char *b, char *result)
```

```
{
```

```
    int i;
```

```
    for(i = 0; i < 8; i++)
```

```
        result[i] =
```

```
            ((a[i] - '0') ^
```

```
            (b[i] - '0')) + '0';
```

```
    result[8] = '\0';
```

```
}
```

```
void incrementCounter(char *counter)
```

```
{
```

```
    int i;
```

```
    for(i = 7; i >= 0; i--)
```

```
    {
```

```
        if(counter[i] == '0')
```

```
        {
```

```
            counter[i] = '1';
```

```

        break;
    }
    else
    {
        counter[i] = '0';
    }
}
}

int main()
{
    char plaintext[25];
    char ciphertext[25];
    char decrypted[25];

    char block[9];
    char stream[9];
    char result[9];
    char counter[] = "00000000";

    char key[] = "0111111101";

    int i;

    printf("Enter 24-bit plaintext: ");
    scanf("%24s", plaintext);

    ciphertext[0] = '\0';

    for(i = 0; i < 24; i += 8)
    {
        strncpy(block, plaintext + i, 8);
        block[8] = '\0';

        sdesEncrypt(counter, key, stream);

        xorBits(block, stream, result);

        strcat(ciphertext, result);

        incrementCounter(counter);
    }

    printf("Ciphertext : %s\n", ciphertext);

    strcpy(counter, "00000000");

```

```

    decrypted[0] = '\0';

    for(i = 0; i < 24; i += 8)
    {
        strncpy(block, ciphertext + i, 8);
        block[8] = '\0';

        sdesEncrypt(counter, key, stream);

        xorBits(block, stream, result);

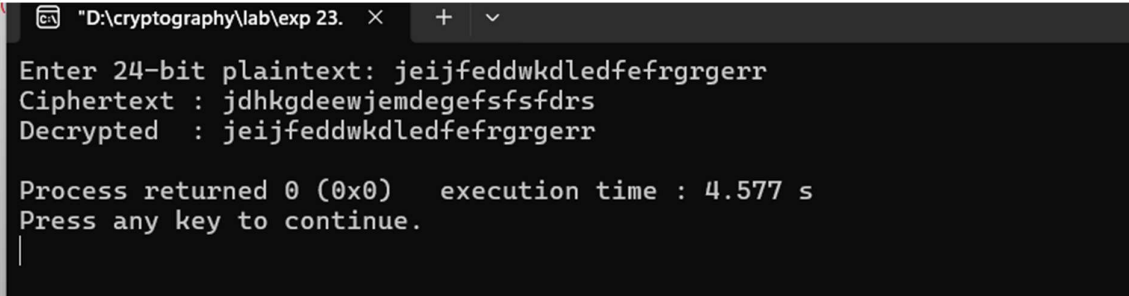
        strcat(decrypted, result);

        incrementCounter(counter);
    }

    printf("Decrypted : %s\n", decrypted);

    return 0;
}

```



24. #include <stdio.h>

```

long long gcd(long long a, long long b)
{
    while(b != 0)
    {
        long long temp = b;
        b = a % b;
        a = temp;
    }

    return a;
}

long long modInverse(long long e, long long phi)
{
    long long t = 0;

```

```

long long newt = 1;
long long r = phi;
long long newr = e;

while(newr != 0)
{
    long long q = r / newr;

    long long temp = t;
    t = newt;
    newt = temp - q * newt;

    temp = r;
    r = newr;
    newr = temp - q * newr;
}

if(r > 1)
    return -1;

if(t < 0)
    t += phi;

return t;
}

int main()
{
    long long e = 31;
    long long n = 3599;

    long long p = 0;
    long long q = 0;
    long long phi;
    long long d;

    long long i;

    for(i = 2; i * i <= n; i++)
    {
        if(n % i == 0)
        {
            p = i;
            q = n / i;
            break;
        }
    }

```

```

    }

    if(p == 0)
    {
        printf("Factors not found\n");
        return 0;
    }

    phi = (p - 1) * (q - 1);

    d = modInverse(e, phi);

    printf("p = %lld\n", p);
    printf("q = %lld\n", q);
    printf("phi(n) = %lld\n", phi);
    printf("Private key d = %lld\n", d);

    return 0;
}

```

```

p = 59
q = 61
phi(n) = 3480
Private key d = 3031

Process returned 0 (0x0)   execution time : 0.787 s
Press any key to continue.
|

```

25. #include <stdio.h>

```

long long gcd(long long a, long long b)
{
    while(b != 0)
    {
        long long temp = b;
        b = a % b;
        a = temp;
    }

    return a;
}

int main()
{
    long long n;
    long long plaintext;

```



```

long long factor;

printf("Enter RSA modulus n: ");
scanf("%lld", &n);

printf("Enter plaintext block: ");
scanf("%lld", &plaintext);

factor = gcd(plaintext, n);

printf("\nGCD(P, n) = %lld\n", factor);

if(factor > 1 && factor < n)
{
    printf("A non-trivial factor of n has been found.\n");
    printf("RSA modulus can be factored.\n");
    printf("p = %lld\n", factor);
    printf("q = %lld\n", n / factor);
}
else
{
    printf("No useful factor of n was found.\n");
}

return 0;
}

```

```

Enter RSA modulus n: 56
Enter plaintext block: froogotgtgk

GCD(P, n) = 8
A non-trivial factor of n has been found.
RSA modulus can be factored.
p = 8
q = 7

Process returned 0 (0x0)   execution time : 5.981 s
Press any key to continue.
|

```

26. #include <stdio.h>

long long gcd(long long a, long long b)

```
{  
    while(b != 0)  
    {  
        long long t = b;  
        b = a % b;  
        a = t;  
    }  
    return a;  
}
```

long long modInverse(long long e, long long phi)

```
{  
    long long t = 0, newt = 1;  
    long long r = phi, newr = e;  
  
    while(newr != 0)  
    {  
        long long q = r / newr;  
  
        long long temp = t;  
        t = newt;  
        newt = temp - q * newt;  
  
        temp = r;  
        r = newr;  
        newr = temp - q * newr;  
    }  
  
    if(r != 1)  
        return -1;  
  
    if(t < 0)  
        t += phi;  
  
    return t;  
}
```

int main()

```
{  
    long long p = 61;  
    long long q = 53;  
    long long n = p * q;  
    long long phi = (p - 1) * (q - 1);
```

```

long long old_d = 2753;
long long new_e = 17;
long long new_d;

printf("Old Private Key = %lld\n", old_d);

new_d = modInverse(new_e, phi);

printf("New Public Key = (%lld, %lld)\n", new_e, n);
printf("New Private Key = (%lld, %lld)\n", new_d, n);

printf("\nNew key is safe only if the new private key is kept secret.\n");
printf("The old leaked private key remains compromised.\n");

return 0;
}

```

```

2 Old Private Key = 2753
2 New Public Key = (17, 3233)
2 New Private Key = (2753, 3233)
2
2 New key is safe only if the new private key is kept secret.
3 The old leaked private key remains compromised.
3
3 Process returned 0 (0x0)   execution time : 0.758 s
3 Press any key to continue.
3

```

27. #include <stdio.h>

```

long long powerMod(long long base, long long exp, long long n)
{
    long long result = 1;

    while(exp > 0)
    {
        if(exp % 2 == 1)
            result = (result * base) % n;

        base = (base * base) % n;
        exp /= 2;
    }

    return result;
}

int main()
{
    long long e, n;

```

```

char message[100];
int i;
long long c;

printf("Enter public exponent e: ");
scanf("%lld", &e);

printf("Enter modulus n: ");
scanf("%lld", &n);

printf("Enter message: ");
scanf("%s", message);

printf("\nEncrypted values:\n");

for(i = 0; message[i] != '\0'; i++)
{
    int m = message[i] - 'a';

    c = powerMod(m, e, n);

    printf("%c -> %lld\n", message[i], c);
}

printf("\nThe method is not secure because identical plaintext\n");
printf("characters always produce identical ciphertext values.\n");

return 0;
}

```

```

Enter public exponent e: 4
Enter modulus n: 89
Enter message: ddejff

Encrypted values:
d -> 81
d -> 81
e -> 78
j -> 64
f -> 2
j -> 64
f -> 2

The method is not secure because identical plaintext
characters always produce identical ciphertext values.

Process returned 0 (0x0)   execution time : 10.413 s
Press any key to continue.

```

28. #include <stdio.h>

long long powerMod(long long base, long long exp, long long q)
{

long long result = 1;

while(exp > 0)

{

if(exp % 2 == 1)

result = (result * base) % q;

base = (base * base) % q;

exp /= 2;

}

return result;

}

int main()

{

long long a, q;

long long xa, xb;

long long A, B;

long long keyAlice, keyBob;

printf("Enter public base a: ");

scanf("%lld", &a);

printf("Enter public prime q: ");

scanf("%lld", &q);

printf("Enter Alice secret x: ");

scanf("%lld", &xa);

printf("Enter Bob secret x: ");

scanf("%lld", &xb);

A = powerMod(a, xa, q);

B = powerMod(a, xb, q);

keyAlice = powerMod(B, xa, q);

keyBob = powerMod(A, xb, q);

printf("\nAlice sends: %lld\n", A);

printf("Bob sends : %lld\n", B);

```

printf("Alice key : %lld\n", keyAlice);
printf("Bob key   : %lld\n", keyBob);

if(keyAlice == keyBob)
    printf("Shared key established successfully.\n");
else
    printf("Key exchange failed.\n");

return 0;
}

```

```

Enter public base a: 5
Enter public prime q: 3
Enter Alice secret x: 5443rfccfbg
Enter Bob secret x:
Alice sends: 2
Bob sends   : 1
Alice key   : 1
Bob key     : 1
Shared key established successfully.

Process returned 0 (0x0)   execution time : 9.796 s
Press any key to continue.

```

29. #include <stdio.h>

```

int main()
{
    int rate = 1024;
    int state = 1600;
    int capacity;
    int lanes;
    int nonzeroRateLanes;
    int zeroCapacityLanes;
    int rounds = 0;

    capacity = state - rate;

    lanes = state / 64;
    nonzeroRateLanes = rate / 64;
    zeroCapacityLanes = capacity / 64;

    printf("State size = %d bits\n", state);
    printf("Rate = %d bits\n", rate);
    printf("Capacity = %d bits\n", capacity);

    printf("Total lanes = %d\n", lanes);
    printf("Non-zero rate lanes = %d\n", nonzeroRateLanes);
}

```

```

printf("Zero capacity lanes = %d\n", zeroCapacityLanes);

while(zeroCapacityLanes > 0)
{
    rounds++;

    zeroCapacityLanes -= nonzeroRateLanes;

    if(zeroCapacityLanes < 0)
        zeroCapacityLanes = 0;
}

printf("Time/steps required = %d\n", rounds);

return 0;
}

```

```

State size = 1600 bits
Rate = 1024 bits
Capacity = 576 bits
Total lanes = 25
Non-zero rate lanes = 16
Zero capacity lanes = 9
Time/steps required = 1

Process returned 0 (0x0)    execution time : 0.720 s
Press any key to continue.
|

```

30. #include <stdio.h>

```

unsigned int xorBlock(unsigned int a, unsigned int b)
{
    return a ^ b;
}

int main()
{
    unsigned int X;
    unsigned int T;
    unsigned int secondBlock;
    unsigned int result;

    printf("Enter one-block message X in hexadecimal: ");
    scanf("%x", &X);

```

```

printf("Enter CBC-MAC T in hexadecimal: ");
scanf("%x", &T);

secondBlock = xorBlock(X, T);

result = xorBlock(X, secondBlock);

printf("\nX = %08X\n", X);
printf("T = %08X\n", T);

printf("Second block = X XOR T = %08X\n",
       secondBlock);

printf("Constructed message:\n");
printf("%08X || %08X\n", X, secondBlock);

printf("\nFinal XOR value = %08X\n", result);

printf("The construction demonstrates the CBC-MAC extension vulnerability.\n");

return 0;
}

```

```

Enter one-block message X in hexadecimal: njnjbbyjjbhj
Enter CBC-MAC T in hexadecimal:
X = 00000000
T = 00000000
Second block = X XOR T = 00000000
Constructed message:
00000000 || 00000000

Final XOR value = 00000000
The construction demonstrates the CBC-MAC extension vulnerability.

Process returned 0 (0x0)   execution time : 3.816 s
Press any key to continue.

```